Removing the Trusted Dealer from Orca's Linear Layers: GPU-Accelerated Ring-LPN Preprocessing — Status and Proposal (v2.3)

Alp

2026-07-29 (v2.3; adds the S1 functionality, transcript, leakage, simulator, and proof-obligation contract of §3.4)

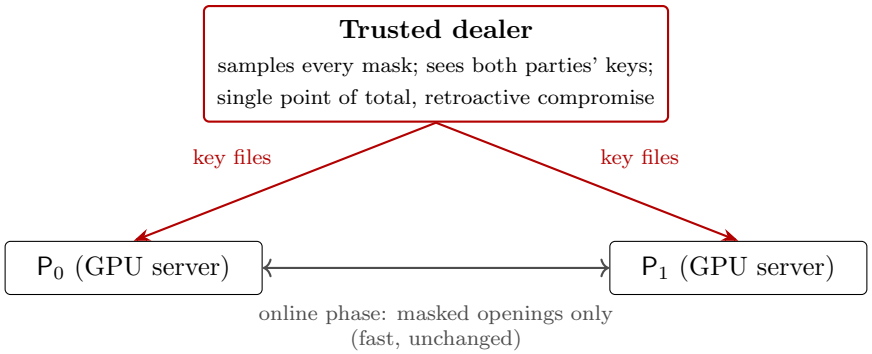
Abstract

Orca performs two-party secure neural-network training and inference on GPUs, but its preprocessing model requires a *trusted dealer*: a third party that generates all correlated randomness and could, if compromised, break the privacy of every past computation at once. This document (1) describes Orca's current preprocessing architecture and the precise sense in which our existing pipeline — which already produces Orca's linear-layer keys from real Ring-LPN correlation generators on GPUs, validated through Orca's unchanged online code — is *not yet* dealerless; (2) motivates removing the dealer with GPU-accelerated pseudorandom correlation generators (PCGs), using measured evidence for where the cost actually lives; (3) proposes a concrete design of five components (distributed key generation, PCG-sourced share conversion, coin-tossed randomness, two-process deployment, and a security parameter audit) and shows how they compose to remove every remaining trust gap for the linear layers; and (4) specifies the experimental setup: platform, validation methodology, the measured baseline, and six milestone experiments, each with a mechanical pass/fail gate. The first design component now has a party-separated host *protocol-logic and compatibility prototype* using ideal OT/triple/OLE functionalities and the unchanged evaluator's explicitly non-cryptographic correctness PRG. It functionally validates 2,432 two-party-generated DPF key pairs across six configurations up to the production tree depth, covers two deterministic point/payload edges, rejects six invalid encodings, and independently corrupts each of the five serialized key-field classes (all controls fail evaluation as required). It also models the distinguishing power of a sign opening removed in v2.2 (§4.3). This is evidence of executable correctness and primitive accounting, not computational privacy or 128-bit security. The S1 contract now fixes the target ideal functionality, exact message transcript, allowed leakage, simulator obligations for either corrupted party, and unresolved proof lemmas; it does not upgrade that security claim. All performance claims are anchored in regenerated CSV artifacts; measured numbers, derived numbers, and estimates are labelled as such throughout.

1 Problem overview: Orca's preprocessing and its trusted dealer

1.1 Background: Orca in one page

Orca [1] is a two-party secure computation (2PC) system for neural-network training and inference, engineered around GPUs. Two servers P_0 and P_1 hold additive secret shares of all inputs, weights, and activations; neither learns anything beyond its own shares. Like most practical 2PC systems, Orca splits work into an input-independent *preprocessing* (offline) phase that generates correlated randomness, and a fast *online* phase that consumes it:



Key types the dealer produces: (a) Beaver-triple matrix keys for *linear* layers — **the subject of this work**; (b) DCF keys for *nonlinear* layers — out of scope here, but addressed by the same distributed key-generation machinery as future work (§3.2).

Figure 1: Orca's preprocessing today. The dealer is trusted with every secret of the offline phase. This proposal removes it for the linear-layer keys; the online phase is never modified.

- **Linear layers** (convolutions, fully-connected/FC layers) consume *Beaver-triple–style matrix keys*: for a layer multiplying an $M \times K$ matrix by a $K \times N$ matrix, the parties need shares of random masks A, B and of the product terms that make the masked multiplication identity work. Online, each party opens masked operands and runs the local computation implemented by `gpuMatmulBeaver`.
- **Nonlinear layers** (ReLU, truncation) consume function secret sharing (FSS) keys — distributed comparison functions (DCFs) — with a different structure.

In stock Orca *both* kinds of keys are produced by a **trusted dealer**: a third machine that samples every mask, computes every correlated value, and ships each party its key files (Figure 1).

1.2 Current status of our pipeline

Over the preceding milestones this project built, and validated on GPUs, a preprocessing pipeline that produces Orca's FC-layer keys from *Ring-LPN pseudorandom correlation generators* rather than from a dealer's coin flips. The pipeline (Figure 2; components defined in §3) already works end-to-end with *real* correlation generators — not idealized stand-ins — at both supported modulus widths, and its output keys are validated by the strongest referee available: Orca's *unchanged* online code reconstructs the correct products (§4.3 gives the measured evidence).

1.3 Why this is not yet dealerless

"The pipeline runs with real generators" is not the same claim as "the dealer is gone." Five gaps remain; each is a precise, code-level statement, and the design in §3 addresses them one for one.

G1 — Centralized SPFSS key generation. The correlation generator compresses each party's sparse noise into sums of distributed point functions (SPFSS; §2.2). In our engine the function `build_spfss_keys()` currently reads *both* parties' noise vectors and writes both key halves — it is a dealer in miniature, marked as an oracle boundary in the source.

G2 — Dealer-supplied conversion correlations. The pipeline computes over prime fields, Orca over $\mathbb{Z}_{2^{bw}}$; the share conversion between them (§3.5) consumes edaBits, daBits and

AND triples that a validated prototype currently takes from a labelled dealer block, and the end-to-end transcript still calls an exact conversion oracle.

G3 — Common-seed randomness. Benchmarks derive *both* parties' randomness from one `mt19937_64` seed. A party knowing the seed can recompute the other party's secret noise outright, so this voids all security independently of every other parameter.

G4 — Unaudited security parameters. The deployed noise parameters (c=2, t=8) are *correctness* parameters for exercising the machinery. No concrete bit-security has been established, and our fully-splitting primes place us in the *splittable* Ring-LPN regime, whose parameters must be derived independently (§3.8).

G5 — Scope: nonlinear layers. ReLU and truncation keys come from the same dealer via a different key type (DCF). They are explicitly out of scope for this proposal; §3.2 states why the same machinery extends to them.

Contributions of this document. (1) A gap analysis (G1–G5) separating what is verified from what is trusted; (2) a design of five components (D1–D5) that remove G1–G4, with the explicit composition argument in Table 4; (3) a *working host protocol-logic prototype of D1*, the distributed key-generation component: the two-party protocol implemented party-separated and functionally validated on 2,432 key pairs by the unchanged existing evaluator, using ideal transport functionalities and a non-cryptographic correctness PRG, with all primitive consumption measured (§4.3); (4) a re-derived, tree-accurate cost model for distributed key generation, including a bootstrap self-sustainability condition ($3c^2t^2 < n$) that couples the security audit to the key-generation design; (5) an experimental plan of six milestone experiments with mechanical pass/fail gates, continuing the artifact discipline under which every number in this document was produced.

2 Motivation: why dealerless, and why on GPUs

2.1 Why remove the dealer

The dealer is the single point of *total, retroactive* compromise: it holds every mask of every computation, so one breach — or one subpoena, or one insider — retroactively breaks the privacy of everything the two parties ever computed. The trust assumption is also operationally awkward: in many two-party deployments (two mutually distrusting companies; a hospital and an ML provider) no mutually trusted third machine exists, and dealer key material for training runs is measured in gigabytes that must be generated, stored, and shipped securely. Removing the dealer replaces an institutional assumption with a cryptographic one.

2.2 Why pseudorandom correlation generators, and which one

A training run consumes correlated randomness for billions of secure multiplications; any protocol paying per-multiplication communication in the offline phase is dead on arrival at that scale. Pseudorandom correlation generators (PCGs) [5, 6] restructure the cost: the parties run one small interactive setup yielding short correlated seeds, then each party *locally* — with zero communication, "silently" — expands its seed into millions of correlated samples.

The atomic correlation we need is *oblivious linear evaluation* (OLE): P_0 holds u, P_1 holds v, and they obtain additive shares of $u \cdot v$. Two OLEs make one Beaver triple; Beaver triples are exactly what Orca's linear-layer keys package. The PCG we build on is the ring-OLE generator of

Boyle et al. [6], which we refer to as the **BCG+20 generator**.[1] In outline, over the negacyclic ring $\mathcal{R}_p = \mathbb{Z}_p[X]/(X^n + 1)$:

1. Each party σ samples c *sparse* secret polynomials $e_1^\sigma, \ldots, e_c^\sigma$ (only t nonzero coefficients each). Its generator input is $x_\sigma = \sum_i a_i e_i^\sigma$ for public random a_i — pseudorandom under the Ring-LPN assumption (§3.8).

2. The target product expands as $x_0 x_1 = \sum_{i,j} a_i a_j \, (e_i^0 \, e_j^1)$: all secrecy concentrates in the c^2 products of sparse polynomials, and *sparse times sparse stays sparse* — each product has at most t^2 nonzeros, at positions that are *sums* of the two parties' secret positions with values that are *products* of their secret values.

3. Compact shares of such sparse vectors are exactly what sums of distributed point functions provide (SPFSS, built from DPF trees [3]): kilobyte-sized keys whose full-domain evaluations sum to the hidden sparse vector.

4. Each party locally evaluates its trees everywhere, multiplies by the public $a_i a_j$ (fast NTT polynomial arithmetic), and accumulates. Output: a *ring OLE*, $z_0 + z_1 = x_0 \, x_1$ in $\mathcal{R}_p$.

Slot packing — the amortization that makes this affordable. Our primes are chosen so that $X^n + 1$ splits into n linear factors mod p; then the negacyclic number-theoretic transform (NTT) is a ring *isomorphism* $\mathcal{R}_p \to \mathbb{Z}_p^n$, and reading one ring OLE per coordinate ("slot") yields up to $n = 8{,}192$ *independent scalar OLEs* from a single ring-OLE expansion. Because the NTT is linear, each party transforms its own shares locally. Measured consequence (§4.3): all linear-layer shapes in our suite consume exactly $2L\lceil MKN/n \rceil$ ring OLEs (L = number of CRT limbs, §2.4) — e.g. **two** ring OLEs for an entire 16×32×16 layer at $Q{=}p_0$, where the same layer would otherwise consume 16,384 per-scalar OLE calls.

2.3 Why GPUs

The motivation is empirical, not aesthetic. Profiling the expansion (§4.3) shows that **SPFSS full-domain evaluation — millions of independent AES/PRG invocations across thousands of DPF trees — dominates the cost** (∼99% at realistic noise weights; the NTT is under 1%). That workload is embarrassingly parallel and exactly GPU-shaped, and it is why our expansion runs in milliseconds on one GPU (13.3 ms at the smoke parameters; 61–881 ms at $t{=}64$ depending on noise regime). The proposed distributed key generation (§3.3) preserves this shape: all trees advance *level-synchronously*, so the depth-14 tree walk has 14 sequential batches independent of how many thousands of trees are in flight. This is not an end-to-end round count: the input adder, correction-word openings, and two-stage payload multiplication add dependency stages to be measured with M1's real transport. Orca itself is GPU-resident; its randomness factory should live on the same hardware, and our keys are written byte-compatibly so Orca's GPU online path runs unmodified.

M, K, N	linear-layer dimensions ($M \times K$ by $K \times N$ matmul)
bw	Orca's ring width; shares live in $\mathbb{Z}_{2^{\text{bw}}}$ (bw ≤ 32 here)
n	ring dimension; $\mathcal{R}_p = \mathbb{Z}_p[X]/(X^n + 1)$, $n = 8192$
p_0, p_1	NTT-friendly 62-bit primes $2^{62} - 6 \cdot 2^{24} + 1$, $2^{62} - 7 \cdot 2^{24} + 1$
L, Q	CRT limb count; composite modulus $Q = p_0 p_1$ (L=1: "q64", Q=p_0; L=2: "q128")
d_{DPF}	DPF tree depth ($d_{\text{DPF}} = \log_2(2n)$ for the production uniform-noise tree)
c, t	Ring-LPN noise parameters: c sparse polynomials, t nonzeros each
λ	symmetric security parameter (AES/OT), $\lambda = 128$

2.4 Notation

3 Proposed design: a fully dealerless linear-layer pipeline on GPUs

3.1 Design overview and principles

Figure 2 shows the complete pipeline. Stages drawn solid are implemented and gate-verified today; the two dashed stages are the oracle boundaries G1 and G2, and the design components D1–D5 below replace them and close G3–G4. Three principles carry over from how the existing artifact was built:

1. **The online path is untouchable.** Keys are written byte-compatibly; Orca's online matmul (`gpuMatmulBeaver`) is both the consumer and the independent validation oracle. (With the integration flag off, baseline Orca is byte-identical — verified.)

2. **Oracle boundaries are explicit.** Every idealized component is marked at the exact line in the source and named in every report; numbers produced with an oracle in the loop are never mixed with protocol-backed numbers in one table column.

3. **Every claim has a mechanical gate.** Components ship with suites that exit non-zero on any failure; one command re-validates the whole chain (§4.2).

3.2 Setting and threat model

Two computationally bounded parties; **semi-honest** (honest-but-curious) adversary corrupting one of them. Assumptions: AES as PRG/PRF (λ=128); security of the OT-extension machinery (Ferret-class [8]); hash-based commitments for coin tossing; and the *splittable* Ring-LPN assumption (§3.8). Out of scope, stated as claims discipline rather than fine print: **malicious security** (consistency checks over OT inputs and openings are known techniques and future work), and **nonlinear layers** (their DCF keys are a second dealer axis; the D1 tree-walk machinery extends to them — larger trees, different payload groups, identical level-synchronous OT pattern — as the natural follow-on project). "Dealerless Orca" without the linear-layer qualifier is not claimable at any milestone in this document.

[1] The construction is specified in Figure 2 of [6]; our codebase and earlier internal reports call the implementation the "Figure-2 engine" after that figure number. The present document uses "BCG+20 generator" to avoid referencing a figure that does not appear in it.

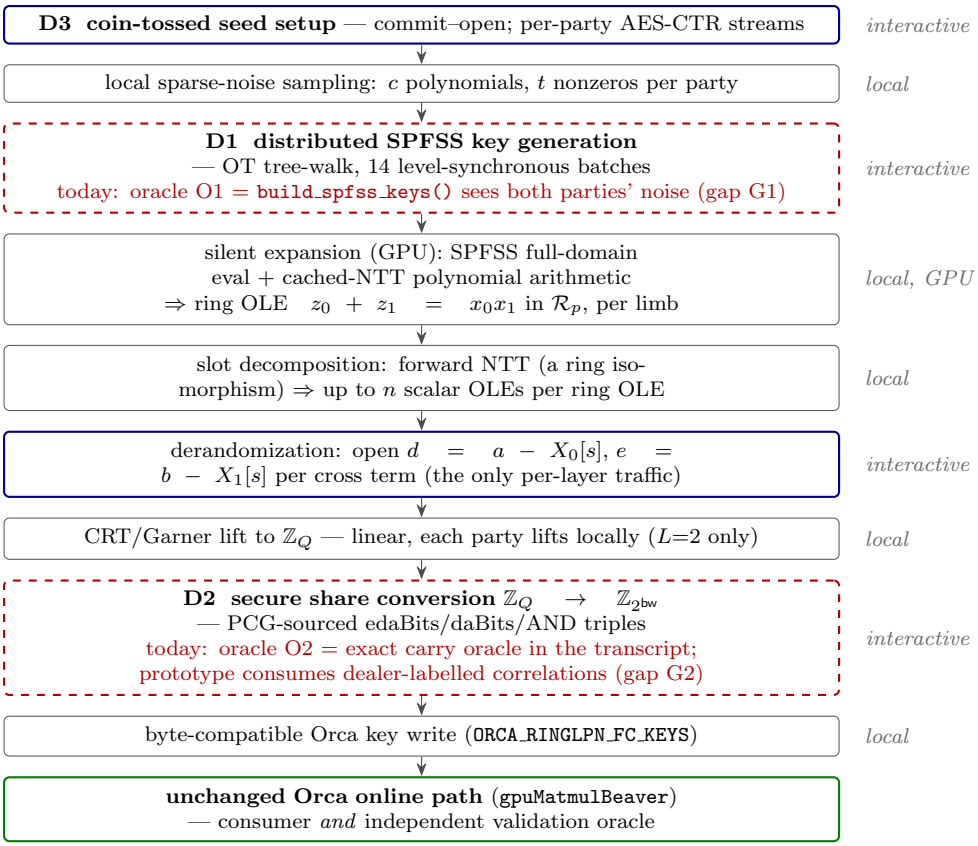


Figure 2: The proposed dealerless pipeline for one party (both parties run it symmetrically; D4 deploys them as two processes over TCP). Solid boxes are implemented and gate-verified today; dashed red boxes are the two oracle boundaries (gaps G1, G2) that components D1 and D2 replace. Blue-framed stages interact over the network; all others are local, and the heavy ones run on the GPU.

Target model versus current D1 evidence. The assumptions above describe the *target* pipeline: AES and real silent-OT/OLE transports. The host artifact of §4.3 instead uses counted ideal OT/triple/OLE interfaces and the unchanged evaluator's splitmix64-based PRG, which its source explicitly labels as non-cryptographic and correctness-only. Accordingly, that artifact establishes protocol-logic compatibility, edge correctness, and primitive counts; it does not establish computational privacy or 128-bit security.

3.3 D1: Distributed SPFSS key generation (removes G1)

What must be computed. For each pair (k, l) of noise nonzeros, the parties must generate DPF keys for the point function with position $\alpha = \mathrm{pos}_k^0 + \mathrm{pos}_l^1$ (a value in $[0, 2n)$: products of degree-$< n$ polynomials have unreduced degree up to $2n-2$; the negacyclic fold $u[i] - u[i+n]$ is linear and applied locally after expansion) and payload $\beta = \mathrm{val}_k^0 \cdot \mathrm{val}_l^1 \in \mathbb{Z}_p$. Observe that the required sharing is *already present in the problem*: α is additively shared (each party knows its own summand) and β is multiplicatively shared, with no extra protocol.

Protocol. We adopt the Doerner–shelat two-party DPF key generation [7] in the variant used by the PCG literature [5, 6]. The parties walk the GGM tree level by level. Off the secret path the two parties' nodes are identical, so XORing each party's aggregate PRG expansions leaves the on-path contribution needed for that level's correction word. Selection by the current secret-shared bit of α uses a 1-of-2 OT on λ-bit strings. We keep control/tag bits separate from the full λ-bit seed state, following the formal BGI construction [4]. We do not derive tags from seed LSBs: the S1 target carries 128 secret seed bits and separate tags.

For the arithmetic payload, let A_b be party b's signed leaf-seed aggregate and F_b its signed control-bit aggregate. A first scalar OLE produces $\gamma_0 + \gamma_1 = \beta_0\beta_1 = \beta$. Set

$$d_0 = \gamma_0 - A_0, \quad d_1 = \gamma_1 - A_1, \qquad s_0 = F_0, \quad s_1 = F_1,$$

so $d_0 + d_1 = \beta - A_0 - A_1$ and $s_0 + s_1 = F_0 + F_1 \in \{+1, -1\}$. Two additional directional OLEs share $d_0 s_1$ and $s_0 d_1$; adding those cross-term shares to the local products $d_0 s_0$ and $d_1 s_1$ yields shares $w_0 + w_1 = (d_0 + d_1)(s_0 + s_1)$. The parties open only this standard public key material, $\mathit{finalCW} = w_0 + w_1$; they never open the d_b, s_b, or their sign. The predecessor transcript opened both F_b values. Although their sign is marginally independent of α, conditioning on one party's leaf control-bit vector makes it select the class containing α, so that opening was invalid.

The input bits of α come from a secure ripple adder ($\log(2n) - 1$ Beaver bit triples). This is the concrete integration choice for arithmetic position shares, not a claim of new ripple-adder research. The known 1-OT-per-level walk optimization remains M1 engineering.

Prototype status (implemented; §4.3). The party-separated host implementation generates standard-format DPF keys from shared (α, β) and passes full-domain validation by the *unchanged* existing evaluator on every tree tested: 2,432 key pairs, both CRT primes, depths through $\log(2n) = 14$. It uses ideal-functionality OT, bit triples, and three scalar OLEs, plus the evaluator's non-cryptographic correctness PRG; these are exactly why the executable is not evidence of computational privacy. At depth 14 it records 28 string OTs, 13 bit triples, 3 scalar OLEs, 1,908 logical opened bits, and 3,816 raw revealed-share bits per tree. Neither bit count is measured real-transport traffic. The old-sign regression reconstructs the removed sign only in the omniscient harness and confirms that it selects a proper leaf-control-bit class containing the true point; this is a regression model of the old leak, not a privacy proof.

3.4 S1 security contract for D1 and FC composition

Two-level ideal functionality and state topology. For a public $(\mathit{sid}, \mathit{tree}, d_{\mathrm{DPF}}, p)$, $\mathcal{F}_{\mathrm{DDPF}}$ receives $(\mathit{off}_b, \beta_b, \mathit{root}_b)$ from P_b, where $0 \le \mathit{off}_b < 2^{d_{\mathrm{DPF}}-1}$, $\beta_b \in \mathbb{Z}_p^*$ is canonically encoded, and root_b is that party's uniform 128-bit random-tape draw. It rejects a malformed or duplicate identifier with the same public abort at both parties before consuming correlation. Otherwise it sets $\alpha = \mathit{off}_0 + \mathit{off}_1$ without modular wrap and $\beta = \beta_0\beta_1 \bmod p$, runs standard DPF generation conditioned on the supplied roots, and returns only K_b to P_b. A batch may contain arbitrarily correlated or repeated private inputs, but every tree uses a distinct identifier, independent roots, and fresh primitive correlation.

The stateful $\mathcal{F}_{\mathrm{FC}}$ takes a public ordered topology table whose entries reference public mask-wire handles. A handle names one $\mathbb{Z}_{2^{\mathsf{bw}}}$ mask and its additive shares; repeated handles intentionally reuse those shares. A training-layer row names $(X, W, Y_{\mathrm{bias}}, V_W, V_Y, G)$, its public truncation/optimizer parameters, and whether momentum is enabled; current `FCLayer` requires bias. Each matrix product creates a fresh product-output handle R_C (source-level `mask_Z`) and splits

$$C_0 + C_1 = AB + R_C \pmod{2^{\mathsf{bw}}}.$$

Forward reads (X, W) and emits $X_b \| W_b \| C_{\mathrm{f},b}$; weight gradient reads the same X plus the incoming gradient handle G and emits $G_b \| C_{\mathrm{dW},b}$; input gradient, when enabled, reads that same G and the same W and emits only $C_{\mathrm{dX},b}$.

The source-level forward transition broadcasts and adds persistent Y_{bias} to $R_{C,\mathrm{f}}$ before truncation; the truncated result is the next activation-mask handle. Backward row-sums G to obtain the bias gradient dY, truncates the dX output, and applies both optimizer transitions $(W, V_W, dW) \mapsto (W', V'_W)$ and $(Y_{\mathrm{bias}}, V_Y, dY) \mapsto (Y'_{\mathrm{bias}}, V'_Y)$. Current Orca initializes $W, Y_{\mathrm{bias}}, V_W, V_Y$ to zero; X comes from the preceding truncated output. These persistent operand/state masks are neither resampled nor assumed independent per invocation. Freshness applies instead to every OT/DPF/OLE/conversion correlation. S7 implements this complete forward/bias/truncation/dW/dX/bias-gradient/dual-optimizer handle and serialization transition before S8 composes its proof; S9 scales and measures that state machine. The present artifact exercises only a forward-shaped matmul.

All ideal calls carry $(\mathit{sid}, \mathit{invocation}, \mathit{tree/slot}, \mathit{phase}, \mathit{ordinal})$ and abort before output on identifier reuse. $\mathcal{F}_{\mathrm{BT}}$ returns private XOR shares of fresh uniform $(a, b, c = a \wedge b)$; the Beaver openings remain real protocol messages. $\mathcal{F}_{\mathrm{OT}}^{128}$ reveals only the selected sender string, and $\mathcal{F}_{\mathrm{OLE}}^{p}$ samples g_0 uniformly and sets $g_1 = x_0 x_1 - g_0$. The composition additionally uses $\mathcal{F}_{\mathrm{COIN}}$ for one uniform public seed, $\mathcal{F}_{\mathrm{RINGOLE}}$ for private shares (X_b, Z_b) with $Z_0 + Z_1 = X_0 X_1$ in the public ring, and $\mathcal{F}_{\mathrm{CONV}}$. For canonical $z_b \in [0, Q)$, the last functionality computes

$$S = z_0 + z_1, \quad \mathit{wrap} = [S \geq Q], \quad v = S - \mathit{wrap}\, Q,$$

then returns uniform additive shares of $v \bmod 2^{\mathsf{bw}}$ and never opens *wrap*. Real transports and realizations of these ideal interfaces are later milestones, not current evidence.

For canonical additive shares, $a_0 + a_1 < 2^{\mathsf{bw}+1}$ and $b_0 + b_1 < 2^{\mathsf{bw}+1}$, so a length-$K$ integer dot product is strictly below $K\, 2^{2\mathsf{bw}+2}$. The FC functionality accepts only public triples satisfying $K\, 2^{2\mathsf{bw}+2} < Q$: reduction modulo 2^{bw} then gives the intended mask product before any reduction modulo Q. This rules out q62/bw32 entirely, caps q62/bw24 at $K \leq 2^{12} - 1$, and is checked before masks are read; q62/bw32 inadmissibility is independently exercised by the executable negative control in `test_orca_zp_bridge`. The current `buildCShare` instead tests the clear, secret-mask-dependent `dot >= Q`; that is a centralized oracle and a forbidden value-dependent abort, not target behavior. Two further security blockers are in the current GPU SPFSS keygen: it uses a Doerner–shelat-style low-bit control encoding [7]; the GPU code masks each seed LSB, leaving 127 secret seed bits, while the centralized kernel derives every root deterministically from one 64-bit `seed_base`. The S1 target uses independent OS-CSPRNG roots and 128 secret seed bits with separately generated tags. S3 must change the PRG/root semantics or lower the security target; this draft makes no 128-bit DPF-security claim.

Leakage and abort contract. Common leakage consists of protocol/session and party-role identifiers; the parameter set, $L, d_{\mathrm{DPF}}, p, n, c, t$, CRT limbs, noise mode, epoch and batch/tree counts; public layer shapes, layouts, invocation kind, wire-handle topology, slot/access order, serialized field lengths, fixed message lengths and dependency schedule; common DPF correction words and *finalCW*; masked openings ϵ, δ, d, e and the specified conversion openings; public coin-toss commitments/reveals and seed; and common accept/abort stage. P_b additionally sees only its own DPF inputs/root draw, retained mask shares, local frontier/noise/output-mask state, sender/receiver OT view, triple/OLE shares, authenticated messages, output DPF keys, and emitted Orca fields. The other inputs, roots, keys, mask shares, hidden leaf-control sign, conversion wrap, unused OT messages, OLE masks, and PCG state remain hidden. Public mask-handle reuse is required topology; reuse of primitive correlation or private random-tape draws is prohibited. External packet

observers, timing/microarchitectural side channels, active security, fairness, and availability remain out of scope.

Exact D1 hybrid transcript. The proof target is the $(\mathcal{F}_{\mathrm{BT}}, \mathcal{F}_{\mathrm{OT}}^{128}, \mathcal{F}_{\mathrm{OLE}}^{p})$-hybrid protocol instantiated by the current source. *Phase A*: for each of the $d_{\mathrm{DPF}} - 1$ carry positions, obtain one fresh bit triple, reveal both parties' shares of the two masked one-bit values (ϵ_j, δ_j), apply the Beaver equation, and update XOR-shared carry and point bits. This exposes $2(d_{\mathrm{DPF}} - 1)$ logical bits as $4(d_{\mathrm{DPF}} - 1)$ transmitted shares. The private position summands are the Ring-LPN noise positions, so uniform-mode sums are triangular; regular-mode offsets give the corresponding public-bucket shift, not a uniform modular point.

Phase B: at level i, each party locally expands its live seeds, XOR-aggregates left/right seed and flag children, and defines $Z_b = S_b^{\mathrm{L}} \oplus S_b^{\mathrm{R}}$. Two directional 128-bit OTs share $a_i(Z_0 \oplus Z_1)$. Parties reveal their shares of the standard seed correction word and two flag correction words, then advance locally. These are exactly $130 d_{\mathrm{DPF}}$ logical opened bits and $260 d_{\mathrm{DPF}}$ raw revealed-share bits.

Phase C: one OLE shares $\gamma_0 + \gamma_1 = \beta_0 \beta_1$. With $d_b = \gamma_b - A_b$ and $s_b = F_b$, two directional OLEs share $d_0 s_1$ and $s_0 d_1$. Each party sets $w_b = d_b s_b + x_b + y_b$ and reveals only w_b, giving the standard $\mathit{finalCW} = w_0 + w_1$. Neither d_b, s_b, nor $s_0 + s_1$ is opened. Thus

$$\begin{aligned} \text{logical opened bits} &= 2(d_{\mathrm{DPF}} - 1) + 130 d_{\mathrm{DPF}} + \lceil \log_2 p \rceil, \\ \text{raw revealed-share bits} &= 4(d_{\mathrm{DPF}} - 1) + 260 d_{\mathrm{DPF}} + 2\lceil \log_2 p \rceil. \end{aligned}$$

D1 batch-simulation lemma at the ideal boundary. Fix either corrupt party, its complete arbitrarily correlated input/root vector, public order and identifiers, and its full ideal output-key vector. Condition each key on the same root supplied by that party. The following algorithms generate the complete hybrid view with the real conditional distribution. This lemma does not yet prove that the emitted pair has the standard DPF distribution or instantiate concrete PRG/transport security.

Corrupt P_0. For each tree, take the root seed and common correction words from K_0 and expand the local frontier. At every carry, sample P_0's three bit-triple shares, compute its actual sent shares of (δ_j, ϵ_j), sample the two reconstructed openings uniformly, and set the honest sent shares to the XOR complements. Apply P_0's real Beaver equation, including the public $\delta_j \epsilon_j$ term, and the carry recurrence. Hidden honest triple shares one-time-pad both openings, so induction from carry zero gives the exact joint carry/point-share view.

At every level, derive P_0's aggregates. In the first OT it is receiver: sample the selected output, uniform under the honest sender's fresh mask. In the second it is sender: sample r_0 and record $(r_0, r_0 \oplus Z_0)$. Compute its seed/flag opening shares with the real equations and set the honest shares to complements of the words fixed by K_0. Finally sample its three OLE outputs (γ_0, x_0, y_0) independently, compute d_0, s_0, w_0, and emit only $w_1 = \mathit{finalCW} - w_0$. This ordering also covers zero intermediate values; it does not infer the hidden sign.

Corrupt P_1. Take the root and common words from K_1. Sample its triple shares and masked openings as above, but apply P_1's Beaver equation without the public $\delta_j \epsilon_j$ term. At each level record its first-OT sender pair $(r_1, r_1 \oplus Z_1)$ and sample its selected second-OT receiver output; derive its own seed/flag shares and complement to the fixed common words. Sample (γ_1, x_1, y_1) from the three uniform P_1 OLE marginals, compute d_1, s_1, w_1, and emit only $w_0 = \mathit{finalCW} - w_1$.

Both simulators enforce a consumed-correlation-ID ledger before every ideal call and proceed tree by tree conditioned on the complete prior state. Therefore repeated/correlated private summands do not invalidate the one-time-pad hybrids. S1 treats local coins as independent con-

sumed random-tape draws and makes the root explicit; S3 must prove a concrete, state-consistent CSPRNG interface. In the end-to-end hybrid, only the honest party's hidden PRG stream may be replaced by ideal random strings; the corrupt party's stream and retained state remain consistent.

ID	Proof obligation	Current status
P-CORR	Keys reconstruct $\beta[x = \alpha]$.	2,432/2,432 executable passes
P-DIST/P-KEY	Couple correction words to standard DPF generation conditioned on both roots; instantiate single-key privacy.	open for S3/S8
P-POS	Prove the exact uniform/regular Ring-LPN exponent distribution.	contract fixed; S2 audit open
P-ADD	Simulate either ripple-adder view with explicit triple/opening equations.	closed in bit-triple hybrid
P-LEVEL	Simulate both OT roles and CW shares conditioned on each full key.	closed in OT hybrid
P-PAYLOAD	Simulate the three-OLE view without the removed sign, including zero intermediates.	closed in OLE hybrid
P-BATCH/P-FRESH	Preserve correlated tree inputs and reject correlation-ID reuse before output.	D1 lemma + executable control
P-RNG/P-PCG	Realize the random-tape interface and prove Ring-LPN OLE at the pinned parameters.	open for S2/S3/S8
P-CONV	Realize exact modulo-Q conversion without revealing wrap.	correctness/count gate; S6 proof open
P-TOPO/P-PROC	Match stateful mask reuse, serialized fields, and a two-process implementation.	contract fixed; S7/S9 open

Table 1: S1 proof contract. Closed rows are hybrid-transcript lemmas only, not a computational-security claim; open rows are binding later-stage gates.

Bootstrapping (and why it is not circular). Key generation consumes three scalar OLEs per DPF tree; expansion produces n slot OLEs per ring OLE. The factory therefore feeds itself precisely when

$$3c^2t^2 \;<\; n, \tag{1}$$

in which case the protocol runs in epochs — a few output slots of each epoch are reserved as the seed OLEs of the next, as in [6] — and only epoch zero needs an external source (a small fixed batch of OT-based Gilboa multiplications). At the smoke parameters $(c, t, n) = (2, 8, 8192)$, key generation consumes 768 scalar-OLE slots and the output/input surplus is $8192/768 = 10.67\times$. Condition (1) *couples the security audit to this design*: if M5 (§4.4) raises t to 64 at $n = 8192$, then $3c^2t^2 = 49{,}152 > 8{,}192$ and the ring dimension must grow with t — consistent with the $n \geq 2^{16}$ parameter points in the literature.

GPU batching. All trees of a generation batch advance level-synchronously: the depth-14 tree walk has 14 sequential batched AES/PRG expansions and OT selections, independent of tree count. The secure ripple adder, correction-word openings, and two-stage payload multiplication introduce additional dependency stages. End-to-end network rounds therefore remain an M1 real-transport measurement, not a current result.

Cost model (per ring-OLE limb). Each DPF tree of depth d costs λd correlated-OT instances; Ferret-class silent OT amortizes each instance to roughly 1–8 bits of real traffic (implementation-maturity range; M1 replaces this estimate with measurements). With uniform noise there are c^2t^2 trees of depth $\log(2n) = 14$. With *regular* noise (one nonzero per width-n/t bucket), the t^2 product

points of each polynomial pair fall on $2t-1$ predictable diagonals, so the same c^2t^2 trees shrink to depth $\log(2n/t)$, grouped as $c^2(2t-1)$ SPFSS instances over the smaller domain.[2]

noise, t (c=2)	DPF trees	depth	OT instances	scalar OLEs	est. silent-OT comm.
uniform, t=8	256	14	458,752	768	0.06–0.46 MB
regular, t=8	256	11	360,448	768	0.05–0.36 MB
uniform, t=64	16,384	14	29,360,128	49,152	3.7–29 MB
regular, t=64	16,384	8	16,777,216	49,152	2.1–17 MB

Table 2: Distributed key-generation cost per ring-OLE limb (derived). $n = 8192$, $\lambda = 128$; depth is $\log_2(2n)$ resp. $\log_2(2n/t)$. Scalar OLEs equal three times the tree count. The silent-OT communication estimate excludes the still-unmeasured real OLE transport; M1 replaces both with measurements. The host prototype of §4.3 measures 2 string OTs per level (twice the table's optimized 1-OT-per-level estimate), depth−1 bit triples, and three scalar OLEs per tree.

3.5 D2: PCG-sourced share conversion (removes G2)

The problem. The pipeline holds additive shares over $\mathbb{Z}_Q$ (prime world, where NTTs exist); Orca needs shares over $\mathbb{Z}_{2^{\mathsf{bw}}}$ (hardware world, where they do not). Reducing both shares mod 2^{bw} is wrong whenever the integer share-sum wrapped past Q — and that *wrap bit* depends on both shares jointly, so computing it requires an actual secure protocol.

The protocol (validated prototype). For canonical $z_b \in [0, Q)$, let $S = z_0 + z_1 < 2Q$ and $\ell = \lceil \log_2 2Q \rceil$ (ℓ=63 at q64, 125 at q128). An ideal edaBit samples uniform $R \in \mathbb{Z}_{2^\ell}$, an independent uniform arithmetic first share, and independent uniform XOR masks for its bits, then gives each party its consistent shares. An ideal daBit samples a random bit, an independent uniform Boolean first share, and an independent uniform $\mathbb{Z}_{2^{\mathsf{bw}}}$ first share, then complements each sharing. Under fresh identifiers: (1) mask and open $A = (z_0 + z_1 + R) \bmod 2^\ell$; (2) a ripple adder recovers Boolean shares of $S = A - R$; (3) a second adder obtains $w = [S \geq Q]$ from the carry of $S + (2^\ell - Q)$; (4) the daBit converts w to arithmetic shares; and (5) each party computes $r_i = (z_i - Qw_i) \bmod 2^{\mathsf{bw}}$. Thus $r_0 + r_1 = ((z_0 + z_1) \bmod Q) \bmod 2^{\mathsf{bw}}$ and w is never opened. The prototype is bit-exact against the all-seeing oracle on 8,516 conversions per configuration, including deterministic integer sums $0, Q-1, Q, 2Q-2$ on both sides of the wrap boundary. Its ripple circuit consumes $2(\ell-1)$ AND triples, $2\ell - 1$ post-mask dependency rounds, $5\ell - 3$ logical opened bits, and $10\ell - 6$ raw revealed-share bits. The post-mask count begins after the masked-A opening, includes the final daBit opening, and is not an end-to-end transport-round measurement.

What D2 changes. Two things. *Sourcing:* AND triples come from a binary PCG (the $\mathbb{Z}_2$ instantiation of our own pipeline, or expand-accumulate codes [9]; Ferret [8] yields them at ∼1 bit amortized), edaBits from random bit-shares by the standard construction [10] ($O(\ell)$ AND triples each, one-time batch), daBits from the same batch. *Depth:* the ripple comparator becomes a parallel-prefix (Rabbit-style [11]) comparison — carries obey an associative generate/propagate rule, so all of them come out of a $O(\log \ell)$-depth tree. Rounds are what network latency multiplies; this swap is what makes the two-process deployment (D4) tolerable over real wires.

[2]The 2026-06-10 draft priced the regular-noise row per SPFSS *instance* ($c^2(2t-1)$ units of depth 14), giving 107,520 OT instances at t=8; the counts below are per DPF *tree*, matching `build_spfss_keys()`, and supersede it. Regular noise's decisive advantage is expansion time and key size (measured 14× and 283→233 kB at t=64), not OT

per $\mathbb{Z}_{2^{bw}}$ output	q64 (ℓ=63)	q128 (ℓ=125)
AND triples = $2(\ell-1)$	124	248
edaBit bits = ℓ	63	125
daBits	1	1
logical opened bits = $5\ell-3$	312	622
raw revealed-share bits = $10\ell-6$	624	1,244
post-mask dependency rounds, ripple = $2\ell-1$	125	249
post-mask dependency rounds, prefix target	$\sim$7	$\sim$8

Table 3: Measured conversion cost per output (`secure_convert_prototype.csv`) with executable transcript accounting, plus the prefix-comparator round target. The earlier 375/747 column mixed the two transmitted shares of the masked-A opening with logical AND/B2A openings; it was not a coherent opening or wire-payload metric. Prefix comparison changes correlation counts and must report its own measured logical/raw split.

3.6 D3: Coin-tossed seeds and transcript hygiene (removes G3)

Replace every common `mt19937_64` stream: *shared, public* randomness (the a_i, slot assignments) is derived from seeds fixed by commit-then-open coin tossing over the existing `SigmaPeer` channel; *private* randomness comes from per-party AES-CTR streams; every derived stream is domain-separated by the (layer, epoch, direction, limb) tuple already used by `mixSeed`. This is bookkeeping, not research — but it is a hard precondition for any security claim, because a party that knows the common seed can recompute the other party's sparse noise and simply read the "hidden" correlations off (zero bits of security regardless of (n, c, t)).

3.7 D4: Two-process deployment (turns the transcript into a protocol)

Today's artifacts separate the parties logically within one process. D4 promotes them to two OS processes on two GPUs over the existing `SigmaPeer` TCP layer (the same one Orca's two-party FC test uses). The wire traffic is fully enumerable, which is the point: the derandomization openings ($2 \cdot 2 \cdot MKN \cdot L$ field words per layer — measured as `opened_words`, e.g. 32,768 words $\approx$ 256 kB for the full-capacity 16×32×16 layer at q64), the per-level OT messages of D1, and the conversion openings of D2 (MN outputs × Table 3; note a layer has MKN cross terms but only MN outputs, so conversion amortizes to $1/K$ of the cross-term work).

3.8 D5: Security parameter audit (removes G4)

The assumption behind step 1 of §2.2 is **Ring-LPN**: $(a, \sum_i a_i e_i + e_0)$ is pseudorandom for sparse secret e_i. Our primes make $X^n + 1$ split *completely* — that is what buys slot packing — but it also hands the attacker structure: a fully split ring has CRT projections onto small quotients, giving decoding attacks unavailable in the irreducible-ring variant. The honest hardness regime is *quasi-abelian decoding* [12]. D5 therefore re-derives parameters rather than copying them: run standard syndrome-decoding/ISD estimators against the splittable regime and pin (n, c, t, p_0, p_1) with estimator transcripts for $\geq$ 128-bit security. Representative literature points (c=4, t in the dozens, $n \geq 2^{16}$ in BCG+20-style analyses) suggest t will land well above the smoke value — which is why §4.3's t=64 measurements are quoted as the realistic anchors, why regular noise (measured 14× cheaper expansion) is the default regime, and why condition (1) must be re-checked at the

volume.

audited parameters. Three engineering notes recorded with reversal conditions: $v_2(p_0-1) = 25$ and $v_2(p_1-1) = 24$ leave headroom to grow n to $\sim 2^{23}$ without re-pinning primes; the 62-bit prime class is what our current cheddar-derived Montgomery NTT supports. We separately evaluated the GPU-NTT Merge and four-step implementations of Özcan–Savaş [13, 14]; that code is an external measured dependency, not this paper's contribution, and its Barrett path did not support our 62-bit primes in the audited revision. If D5 re-pins primes at ≤ 60 bits, the backend decision and source/license audit re-open.

This audit is the first implementation gate, not a final-paper-only check: changing the pinned parameters changes GPU memory, NTT support, DPF tree counts, communication, and whether the bootstrap inequality holds. M1 and M3 performance work starts only after M5 pins those values.

3.9 How the components compose to "dealerless"

gap	removed by	verified at (gate)	claim unlocked
G1 centralized keygen	D1	M1, M2	*after M2:* keys from a two-party protocol whose only idealized part is the conversion
G2 conversion correlations	D2	M3	conversion protocol-backed end-to-end
G3 common-seed RNG	D3	M4 (audited in review)	transcript-hygiene preconditions met
(process/state separation)	D4	M4	two-process stateful implementation transcript complete; scoped security claim waits for S8 review
G4 unaudited parameters	D5	M5/S2	pinned concrete-security claim; required before M1/M3 performance work
G5 nonlinear layers	out of scope	—	never claimed here; D1 machinery extends (future work)

Table 4: Composition of the design: every gap of §1.3 maps to a component and to the milestone experiment (§4.4) whose mechanical gate verifies its removal.

3.10 Per-layer cost model (worked example)

For one FC layer (M, K, N) at L limbs, ring dimension n: ring OLEs $= 2L\lceil MKN/n \rceil$; derandomization traffic $= 2 \cdot 2 \cdot MKN \cdot L$ field words; conversions $= MN$ (each per Table 3); key-generation OT material per limb per Table 2; online phase unchanged (byte-compatible keys). Worked example, 16×32×16 at q64 (L=1, bw=16): $MKN = 8192$ cross terms per direction $\Rightarrow$ **2 ring OLEs**; 32,768 opened words $\approx$ 256 kB; $MN = 256$ conversions $\Rightarrow$ 31,744 AND triples ($\sim$4 kB of silent-OT traffic) and 256 daBits; keygen OT material 0.05–0.46 MB (Table 2, t=8). Every number above is either measured in the suite CSVs or derived from a formula stated here.

4 Experiment setup

4.1 Platform and environment

All measurements in this document were produced on one host: 4× NVIDIA RTX 5000 Ada (compute capability 8.9), CUDA 12.6, Linux. Builds run either on the host (`nvcc` on `PATH`) or in the project's `orca-dev` container; two-party tests run as two processes pinned to distinct GPUs via `CUDA_VISIBLE_DEVICES`, communicating over TCP loopback through Orca's `SigmaPeer` layer. The GPU NTT backend is a cheddar-derived merged-stage kernel family (signed Montgomery arithmetic; q32/q64/q128-CRT; negacyclic) with the Hadamard product adaptively fused into the inverse-NTT load and forward NTTs of fixed public vectors cached across expansion iterations.

4.2 Validation methodology

Three rules, applied to every artifact. (1) *Independent oracles*: fast implementations are validated against separately written references — the GPU NTT against a host reference (`host_polymul_reference`), the generator's internals against host suites (135/57/36 host validation cases), the end-to-end pipeline against Orca's *unchanged* online code reconstructing the correct product. (2) *Party-separation discipline*: transcript artifacts compute each party's view in separated state; any read across the boundary is an explicitly named oracle. (3) *Mechanical gates*: every artifact ships source + build script + run script + CSV/MD/log; suites exit non-zero on any failure; one command re-validates every claim in this document (∼15 min on one GPU):

```
RUN_GPU_SMOKE=1 REQUIRE_GPU_SMOKE=1 scripts/run_paper_checkpoint_smoke.sh
```

which must print `ALL GATES PASS`.

4.3 Verified baseline (measured, 2026-06-10 artifacts)

End-to-end real-generator transcript. The flagship suite[3] (9 cases) drives the full pipeline of Figure 2 with the real BCG+20 generator — oracles only at O1/O2 — and validates through unchanged `gpuMatmulBeaver` at q64 ($\mathsf{bw} \leq 16$) and q128 ($\mathsf{bw} = 32$, two CRT limbs, per-party Garner lift), under uniform and regular noise: 9/9 pass.

case	ring OLEs	ideal-OLE equiv.	slots used	opened $\mathbb{Z}_p$ words	contract
q64, 16×32×16, bw=16	2	16,384	8,192	32,768	pass
q128, 16×16×16, bw=32	4	8,192	4,096	32,768	pass

Table 5: Representative rows of the flagship suite (`orca_fc_real_ole_transcript_transcript_suite.csv`). "Ideal-OLE equiv." is the number of per-scalar OLE calls the same layer consumed before slot packing.

Distributed key-generation protocol-logic prototype (corrected 2026-07-29). The artifact[4] implements the two-party key-generation protocol of §3.3 party-separated on the host. Cross-party values flow only through counted ideal functionalities (1-of-2 string OT, Beaver bit triples, and three scalar OLEs) or explicit openings. It emits the standard existing `spfss_host::DPFKey`

[3] Artifact `bench_orca_fc_real_ole_transcript`; per-case results in Table 5's source CSV.

[4] `test_distributed_dpf_keygen`; per-case results in `distributed_dpf_keygen_prototype.csv`.

format, consumed by the untouched `spfss_host::dpfEvalAll`. That evaluator uses a splitmix64 correctness PRG explicitly labelled non-cryptographic; the artifact is therefore a protocol-logic and compatibility prototype, not a secure/private implementation.

The suite (Table 6) generates 2,432 key pairs across six configurations, tree depths 4–14, and both CRT primes. Every pair full-domain-evaluates to $\beta \cdot [x{=}\alpha]$. In every configuration, the first two trees deterministically cover $\alpha = 0$ and $\alpha = 2^{d_{\mathrm{DPF}}} - 2$, factors 1 and $p-1$, and resulting payloads $p-1$ and 1; the remaining trees use random nonzero payload factors. Eight centralized references pass through the same evaluator; five independent root-seed, seed-CW, left-flag-CW, right-flag-CW, and final-CW corruptions fail; and six invalid point/payload encodings abort before correlation is consumed. The field `old_sign_opening_leak_control=yes` records that the omniscient regression model reconstructed the removed sign and observed a proper party-0 leaf-control-bit class containing the true point. This demonstrates the old transcript's distinguishing power; it is not a privacy proof. Every row also requires ideal-mask-draw accounting and a duplicate-correlation-ID negative control to pass. The gate is wired into the repository's one-command re-validation (§4.2).

prime	depth	trees	pass	string OTs	triples	OLEs	logical open	revealed shares
p_0	4	512	512/512	8	3	3	588	1,176
p_0	8	512	512/512	16	7	3	1,116	2,232
p_0	11	384	384/384	22	10	3	1,512	3,024
p_0	14	256	256/256	28	13	3	1,908	3,816
p_1	14	256	256/256	28	13	3	1,908	3,816
p_1	8	512	512/512	16	7	3	1,116	2,232

Table 6: Distributed DPF key-generation host results.

Source: `distributed_dpf_keygen_prototype.csv`. For tree depth d_{DPF}, per-tree counts match $2d_{\mathrm{DPF}}$ string OTs, $d_{\mathrm{DPF}} - 1$ bit triples, three scalar OLEs, $2(d_{\mathrm{DPF}} - 1) + 130d_{\mathrm{DPF}} + \lceil \log_2 p \rceil$ logical opened bits, and $4(d_{\mathrm{DPF}} - 1) + 260d_{\mathrm{DPF}} + 2\lceil \log_2 p \rceil$ raw revealed-share bits. Every row reports `transcript_accounting=pass`, `ideal_mask_draw_accounting=pass`, and `correlation_reuse_control=pass`. Timings are approximately 2.0 ms per depth-14 tree, single-threaded and including validation; they are run observations, not transport measurements. Table 6 proves functional compatibility, edge correctness, and ideal-primitive accounting, not computational privacy or 128-bit security.

Performance anchors. Ring-OLE expansion (n=8192, c=2): 13.3 ms (q64) and 26.8 ms (q128) at the t=8 smoke point; at t=64: 881 ms uniform vs. **61 ms regular** (14×), with pair-key bytes 283 kB vs. 233 kB. SPFSS full-domain evaluation dominates (∼99%); the NTT is under 1% — measured directly, and confirmed by the fact that a 2–8% NTT-side improvement left expansion time unchanged. Conversion prototype: bit-exact on 8,516 conversions per configuration at the costs of Table 3. Byte-compatibility: with the integration flag off, baseline Orca output is byte-identical (verified through the real two-party FC test).

4.4 Planned experiments: six milestones with mechanical gates

M5 — Security parameter memo (first gate). Implements D5. *Gate:* pinned (n, c, t, p_0, p_1) with estimator evidence for $\geq$ 128-bit splittable Ring-LPN security; headline tables regenerated at the pinned parameters; condition (1), NTT support, and GPU memory re-checked.

M1 — Distributed DPF keygen prototype (GPU-batched). Implements D1. *Gate:* for random additively shared α and multiplicatively shared β, the generated key halves full-domain evaluate to the point function for every tree in a batch of $\geq$ 256. Preserve the

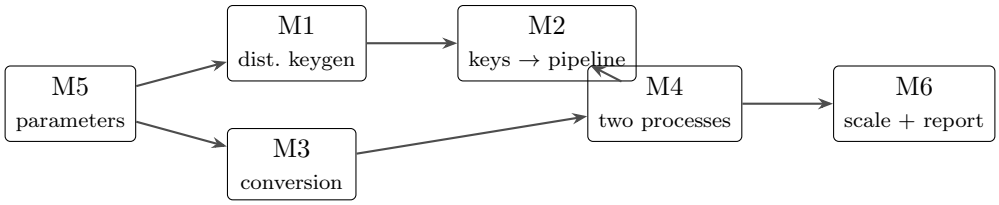


Figure 3: Binding milestone dependencies. M5 pins the exact parameter regime before M1/M3 performance implementation. M1 and M3 may then proceed in parallel; M2 and M3 both gate M4, which gates M6.

`gpuDpfZpFullEvalSum` API and key byte layout while correcting its internal seed semantics to 128 secret bits with separate tags; independently validate the centralized and distributed generators against CPU/GPU evaluators, including seed-LSB-one controls. *Metrics:* real OT/OLE bytes, wall-clock per batch, and end-to-end network rounds. *Status:* the host artifact of §4.3 discharges only the protocol-logic and host-format functional slice (batches ≥ 256, depth 14, unchanged host evaluator). M1 still requires corrected full-128-bit AES/CSPRNG GPU semantics, real silent-OT/OLE transport, GPU level-synchronous batching, compatible GPU keys, and measured bytes and rounds.

M2 — Real-generator transcript driven end-to-end by M1 keys. *Gate:* the existing 9/9 suite passes with `build_spfss_keys()` replaced by the two-party protocol; the only remaining oracle is the conversion. (Because the key format is unchanged, any new failure localizes to the new keygen.)

M3 — PCG-sourced conversion + prefix comparator. Implements D2. *Gate:* the conversion suite (`test_secure_convert`) passes bit-exact with correlations consumed from the binary PCG instead of the dealer block; measured round count drops to $O(\log \ell)$ (Table 3).

M4 — Two-process deployment over TCP. Implements D4 (with D3's coin-tossed seeds). *Gate:* the full pipeline (M1 keygen → expand → derandomize → convert → key write) runs as two OS processes on two GPUs; the `gpuMatmulBeaver` contract passes; one complete forward/bias/truncation/dW/dX/bias-gradient/dual-optimizer sequence matches Orca's persistent mask/velocity handles, reduced matmul fields, and next-state transitions; communication is measured and reported per layer.

M6 — Scale and report. *Gate:* dealerless preprocessing for the FC layers of one Orca model, with separate trusted-dealer, oracle-backed, and protocol-backed columns plus one closest dealerless-preprocessing baseline selected by S2's novelty audit. Compare time, communication, rounds, and key bytes at matched assumptions/shapes; if compatible baseline code is unavailable, give a reproducible normalized comparison and explain why.

4.5 Reporting and claims discipline

The claims ladder is part of the experimental design. *Already claimable (from §4.3)*: "the distributed key-generation protocol logic is implemented party-separated and functionally validated by the unchanged evaluator, using ideal OT/triple/OLE functionalities and a non-cryptographic correctness PRG." This is deliberately not a claim of computational privacy, 128-bit security, M1 completion, GPU byte compatibility, real transports, or two-process isolation. *After M2*: "linear-layer

Beaver keys are generated by a two-party protocol whose only idealized component is the share conversion." *After M4*: only "the full protocol-backed preprocessing transcript runs as two processes"; the security claim remains blocked. *Only after M5/S2 parameters and the S8 proof/implementation review*: "Orca FC preprocessing is dealerless (two-party, semi-honest, splittable Ring-LPN, at the pinned parameters)." *Never claimable within this proposal's scope*: malicious security; any concrete security level before M5; "dealerless Orca" without the linear-layer qualifier. Every measured table in the final report carries the regenerating command; oracle-backed and protocol-backed numbers never share a column; decisions carry written reversal conditions (e.g., the NTT-backend choice re-opens if M5 re-pins primes below 61 bits).

5 Related work

Orca [1] defines the host system: FSS-based 2PC ML on GPUs with a dealer-based preprocessing model. Beaver's technique [2] underlies the linear-layer keys. The PCG line — silent OT extension and PCGs from LPN [5], ring-OLE PCGs from Ring-LPN [6] — provides our correlation generator; Ferret [8] and Silver [9] provide practical silent OT/VOLE for D1's tree-walk material and D2's binary triples. DPFs originate in the FSS line [3]; distributed DPF key generation without a dealer is due to Doerner and shelat [7]. Mixed arithmetic–binary conversions use edaBits/daBits [10] and Rabbit-style comparison [11]. The concrete hardness of our fully-split (splittable / quasi-abelian) Ring-LPN regime is analyzed by Bombar et al. [12]. The active GPU polynomial backend is cheddar-derived; GPU-NTT's Merge and four-step algorithms are separate upstream work by Özcan, Javeed, and Savaş [13, 14]. We use that work only as an external measured comparison/dependency. Its algorithms, implementation, and performance are not contributions of this paper.

6 Summary

Orca's linear-layer preprocessing already runs, measured and gate-verified, from real Ring-LPN correlation generators on GPUs — with exactly two idealized components left in that pipeline (key generation and share conversion) and three hygiene/audit obligations (seeds, deployment, parameters). D1 now has a host protocol-logic and compatibility prototype: 2,432 standard key pairs pass the unchanged evaluator with edge, reference, corruption, and old-leak regression controls, using ideal transport functionalities and a non-cryptographic correctness PRG. This is research progress suitable for evaluating the M1 design, not evidence that M1 or computational privacy is complete. The S1 contract fixes the ideal functionality, exact transcript, leakage, simulators, and open proof obligations; it deliberately does not claim those obligations are discharged. D1–D5 remove the implementation boundaries along M1–M6. Only after the parameter gate and S8 proof/implementation review also close is "dealerless Orca linear-layer preprocessing (two-party, semi-honest, splittable Ring-LPN, at the pinned parameters)" an honest, reproducible claim.

References

[1] N. Jawalkar, K. Gupta, A. Basu, N. Chandran, D. Gupta, R. Sharma. *Orca: FSS-based Secure Training and Inference with GPUs*. IEEE S&P 2024.

[2] D. Beaver. *Efficient Multiparty Protocols Using Circuit Randomization*. CRYPTO 1991.

[3] E. Boyle, N. Gilboa, Y. Ishai. *Function Secret Sharing*. EUROCRYPT 2015.

[4] E. Boyle, N. Gilboa, Y. Ishai. *Function Secret Sharing: Improvements and Extensions*. CCS 2016, pp. 1292–1303. doi:10.1145/2976749.2978429.

[5] E. Boyle, G. Couteau, N. Gilboa, Y. Ishai, L. Kohl, P. Scholl. *Efficient Pseudorandom Correlation Generators: Silent OT Extension and More*. CRYPTO 2019.

[6] E. Boyle, G. Couteau, N. Gilboa, Y. Ishai, L. Kohl, P. Scholl. *Efficient Pseudorandom Correlation Generators from Ring-LPN*. CRYPTO 2020; corrected full version, IACR ePrint 2022/1035 (2022), §5.2 and Remark 5.1.

[7] J. Doerner, a. shelat. *Scaling ORAM for Secure Computation*. CCS 2017.

[8] K. Yang, C. Weng, X. Lan, J. Zhang, X. Wang. *Ferret: Fast Extension for Correlated OT with Small Communication*. CCS 2020.

[9] G. Couteau, P. Rindal, S. Raghuraman. *Silver: Silent VOLE and Oblivious Transfer from Hardness of Decoding Structured LDPC Codes*. CRYPTO 2021.

[10] D. Escudero, S. Ghosh, M. Keller, R. Rachuri, P. Scholl. *Improved Primitives for MPC over Mixed Arithmetic-Binary Circuits*. CRYPTO 2020.

[11] E. Makri, D. Rotaru, F. Vercauteren, S. Wagh. *Rabbit: Efficient Comparison for Secure Multi-Party Computation*. FC 2021.

[12] M. Bombar, G. Couteau, A. Couvreur, C. Ducros. *Correlated Pseudorandomness from the Hardness of Quasi-Abelian Decoding*. CRYPTO 2023.

[13] A. Ş. Özcan, E. Savaş. *Two Algorithms for Fast GPU Implementation of NTT*. Cryptology ePrint Archive, Paper 2023/1410, 2023.

[14] A. Özcan, A. Javeed, E. Savaş. *High-Performance Number Theoretic Transform on GPU Through radix2-CT and 4-Step Algorithms*. IEEE Access 13 (2025), 87862–87883. doi:10.1109/ACCESS.2025.3570024.